ATSEP CONTROL LAB – BANCO FÍSICO

Guía de laboratorio: banco básico de controles con ESP32

Pulsadores, interruptores, una compuerta Y y tres LEDs

Básico ATSEP - Sistemas de Energía - Complemento del *Laboratorio Interactivo de Controles Eléctricos*

Contenido académico y dirección docente: Marco Teran

Introducción

Este laboratorio presenta, desde cero, un **circuito de control eléctrico**: un conjunto de entradas (botones y selectores), reglas de decisión y salidas. En un tablero real, el operador puede dar la orden de **MARCHA**, detener un equipo con **PARO**, observar alarmas y seleccionar una fuente de alimentación. Aquí representaremos esas acciones con señales seguras de $3{,}3\,\mathrm{V}$, sin conectar equipos de potencia.

El banco se construye paso a paso. Primero aprenderá a encender un LED, después a leer pulsadores, luego a mantener una orden de marcha y finalmente a simular una transferencia entre **red eléctrica** y **planta de respaldo**. El LED verde representará el mando de KM1 (red), el naranja el de KM2 (planta) y el rojo una alarma. **Un LED encendido representa una orden, no confirma que exista tensión real ni que haya cerrado un contactor.**

Conozca los componentes antes de conectarlos

ESP32 (microcontrolador).
Es una pequeña computadora programable dentro de una tarjeta llamada *DevKitC*. Ejecuta instrucciones, lee niveles eléctricos mediante sus pines **GPIO** (entradas y salidas de propósito general) y activa salidas. Recibe el programa del computador mediante el cable USB. En este laboratorio funciona como el *controlador* del sistema: interpreta MARCHA, PARO y los selectores, y genera CMD1 y CMD2.

74HC08 (circuito integrado).
Es un chip de 14 patas que contiene **cuatro compuertas lógicas Y (AND)**. Cada compuerta tiene dos entradas y una salida: la salida vale 1 únicamente cuando **ambas entradas valen 1**. Utilizaremos dos compuertas para permitir una orden solo cuando PARO esté liberado. No es un dispositivo de seguridad certificado ni un enclavamiento entre KM1 y KM2.

Protoboard (placa de pruebas).
Permite armar circuitos sin soldadura. Los agujeros de cada grupo de cinco se comunican internamente; los dos lados del canal central están separados. Los rieles laterales pueden estar **interrumpidos** en algunos modelos: verifíquelos con el multímetro antes de usarlos.

Resistencias, LED, pulsadores y DIP.
La resistencia limita corriente o fija una entrada digital. Un LED es un indicador luminoso con polaridad. Un pulsador solo cambia su contacto mientras se oprime; el DIP es un conjunto de pequeños interruptores que conservan su posición.

Palabras clave: `3V3` = alimentación positiva de $3{,}3\,\mathrm{V}$; `GND` = referencia eléctrica o 0 V; `GPIO` = pin digital; `1` = nivel alto, próximo a 3,3 V; `0` = nivel bajo, próximo a 0 V; `CMD` = orden del controlador; `KM1/KM2` = nombres didácticos de los mandos de dos contactores.

Seguridad y cuidado del equipo

Trabaje únicamente con el banco alimentado por USB y a $3{,}3\,\mathrm{V}$. **No conecte la red eléctrica, una planta real, contactores ni tensiones externas a la protoboard.** No aplique 5 V a los GPIO del ESP32. Desconecte el USB antes de mover cables y solicite revisión docente antes de energizar un montaje nuevo.

Objetivo y ruta de trabajo

Simular, construir y comprobar un control básico, desde una salida sencilla hasta una transferencia red–planta, explicando qué corresponde al programa y qué corresponde al cableado.

Momento	Minutos	Aprendizaje
Preparación y reconocimiento	20	Identificar componentes, alimentación y pines
Práctica 1: LED	25	Corriente, resistencia y medición
Práctica 2: entradas	35	Pulsador, interruptor, pull-up y pull-down
Práctica 3: marcha–paro	45	Retención e inhibición por compuerta AND
Práctica 4: red–planta	40	Temporización y transferencia sin solape normal
Cierre y evidencia	15	Explicar, documentar y entregar
Total	**180**	**3 horas**

Materiales de la práctica de laboratorio

Cant.	Pieza	Para qué	Práctica
1	ESP32 DevKitC + cable USB	Programa, lee las entradas y da las órdenes	todas
1–2	Protoboard(s), preferible 830 puntos o dos de 400	Espacio suficiente para ESP32 y el 74HC08	todas
2	Pulsadores de 4 patas (6×6 mm)	MARCHA y PARO	2–4
1	DIP switch de 3 o 4 posiciones	A (red apta), B (planta disponible), FALLA	2–4
1	Integrado 74HC08 (cuatro compuertas Y)	Hace cumplir el PARO por cable	3–4
1	Condensador de $100\,\mathrm{nF}$	Estabiliza la alimentación del 74HC08	3–4
3	LEDs de 5 mm: rojo, verde, naranja (o amarillo)	Alarma, KM1 (red), KM2 (planta)	1–4
3	Resistencias de $330\ \Omega$ (naranja-naranja-marrón-oro en 4 bandas)	Limitan la corriente de cada LED	1–4
7	Resistencias de $10\,\mathrm{k}\Omega$ (marrón-negro-naranja-oro en 4 bandas)	Fijan el valor de cada entrada	2–4
30–40	Cables de conexión (jumpers)	Cableado del banco; detalle en la hoja auxiliar	todas
1	Multímetro	Medir tensiones	1–2

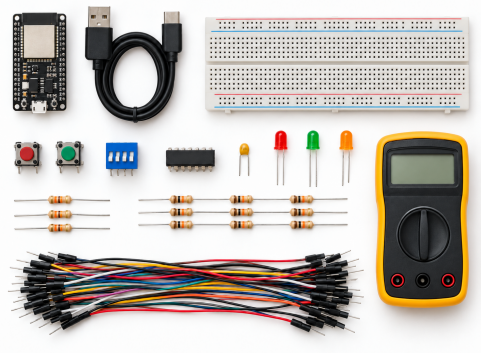


Figura 1: Elementos del kit: fotografía si está disponible; en otro caso, ilustración orientativa (no sustituye la revisión de los componentes reales).

1 Preparación

Objetivo

Reconocer el ESP32, comprobar los puntos de alimentación y preparar el programa antes de conectar las entradas y las salidas.

1. **Revise el kit** (Figura 1) y localice en su tarjeta los rótulos `3V3`, `GND` y `GPIO`. El orden de pines puede cambiar entre fabricantes y versiones; guíese siempre por la serigrafía de **su** tarjeta, no por una foto genérica.
2. **Instale y programe el ESP32.** Con el cable USB conecte la tarjeta al computador y abra el IDE de Arduino. Cree un sketch nuevo, copie el código del **Anexo B** y guárdelo como `banco_simple_esp32.ino`. Seleccione la placa y el puerto correctos, cargue el programa y abra el *Monitor serie* a **115200 baudios**. Debe leer: *«Banco básico ATSEP. Escriba 1, 2, 3 o 4.»*
3. **Desconecte el USB** antes de cablear. Una el pin `3V3` al riel positivo y `GND` al negativo. Compruebe con el multímetro qué segmentos del riel están realmente conectados y únalos mediante jumpers si están interrumpidos.
4. **Trabaje por etapas:** no conecte toda la práctica de una vez. Compruebe cada bloque y solicite revisión antes de energizar el siguiente. Las figuras finales 8 y 9 son referencias globales; para cada señal siga también la tabla completa de conexiones del Anexo A.

Pin ESP32	Conectado a	Práctica
GPIO 18	LED rojo (alarma)	1–4
GPIO 14	Pulsador MARCHA	2–3
GPIO 13	Pulsador PARO y `PARO_OK` (AND)	2–4
GPIO 32, 33, 25	DIP: A, B, FALLA	2–4
GPIO 23	Orden CMD1 → 74HC08 pin 1	3–4
GPIO 22	Orden CMD2 → 74HC08 pin 4	4
`3V3` / `GND`	Rieles rojo y azul	0

2 El LED y su resistencia

Objetivo

Calcular la resistencia de un LED, montarlo y comprobar el cálculo con el multímetro.

Un **LED** (diodo emisor de luz) deja circular corriente principalmente en un sentido: del **ánodo** (+) al **cátodo** (−). Por eso importa cómo se orienta (Figura 2). Tampoco limita por sí mismo la corriente: conectado directamente a un pin puede dañarse o dañar el ESP32. Una **resistencia en serie** controla la corriente y protege el montaje.

El cálculo

La resistencia **se queda** con la tensión que sobra: la del pin menos la del LED (V_F, unos $2{,}0\,\mathrm{V}$ en un LED rojo). Para una corriente cómoda de $4\,\mathrm{mA}$:

$$R = \frac{V_{pin} - V_F}{I_{LED}} = \frac{3{,}3\,\mathrm{V} - 2{,}0\,\mathrm{V}}{0{,}004\,\mathrm{A}} = 325 \;\Rightarrow\; \text{valor comercial: } 330$$

Con $330\,\Omega$, la corriente estimada a partir de una salida ideal de 3,3 V es $(3{,}3 - 2{,}0)/330 \approx 3{,}9\,\mathrm{mA}$: el LED se ve bien y el pin trabaja con una corriente moderada. En las salidas del 74HC08 la corriente real dependerá de su tensión de salida bajo carga; compruébela en el montaje. La resistencia disipa apenas $P = I^2R \approx 5\,\mathrm{mW}$ (soporta $250\,\mathrm{mW}$).

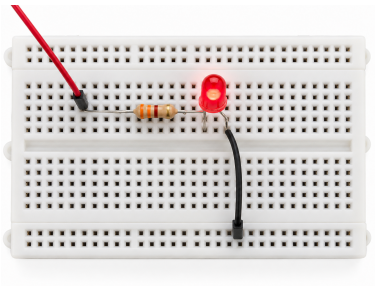


Figura 2: Polaridad del LED antes del montaje: ánodo (+) hacia la resistencia y cátodo (−) hacia GND. La ilustración se emplea si no se encuentra la fotografía.

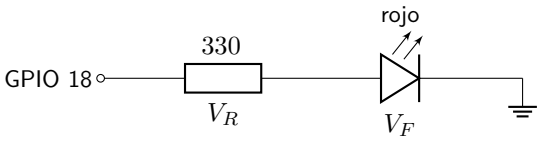


Figura 3: LED rojo de alarma. La pata **larga** del LED va hacia la resistencia; la corta, hacia `GND`.

1. Con el USB desconectado, arme la Figura 3.
2. Conecte el USB y escriba `1` en el Monitor serie: el LED rojo queda encendido.
3. Mida con el multímetro **en voltios de continua**, colocando las puntas a ambos extremos de cada elemento, sin abrir el circuito. Primero mida V_R en la resistencia y luego V_F en el LED. Anote. **No ponga el multímetro en modo corriente directamente entre 3V3 y GND.**

V_R medida	V_F medida	Corriente $I = V_R/330$	¿Coincide con el cálculo?

Pregunta. ¿Cuánto suman $V_R + V_F$? ¿Por qué ese número?

3 Las entradas: pulsador e interruptor

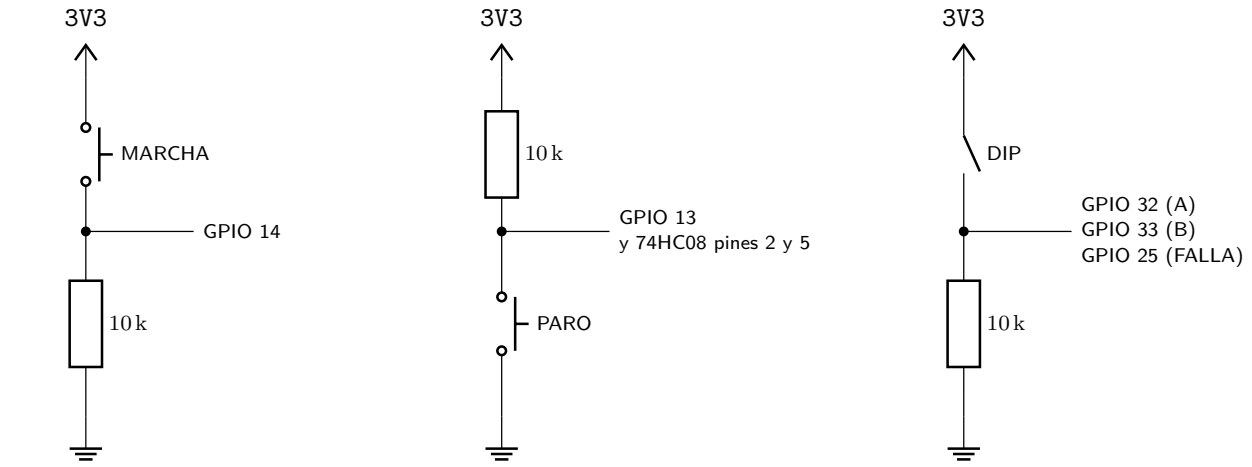
Objetivo

Conectar las cinco entradas sin dejar ninguna al aire y distinguir una entrada momentánea de una mantenida.

Una **entrada digital** interpreta una tensión como 0 o como 1. Si el pin queda desconectado, **flota**: puede indicar 0 o 1 de forma imprevisible, debido a interferencias. La prueba con el dedo ilustra el problema, pero **no garantiza** que se produzcan cambios en todos los montajes. Por eso **cada entrada lleva una resistencia de** $10\,\mathrm{k}$ que fija su valor mientras el contacto está abierto. En este banco hay dos formas:

- **Pull-down** (resistencia a `GND`): abierto lee 0; al cerrar el contacto hacia `3V3` lee 1. Se usa en MARCHA, A, B y FALLA.
- **Pull-up** (resistencia a `3V3`): abierto lee 1; al cerrar el contacto hacia `GND` lee 0. Se usa en PARO: en reposo vale 1 (`PARO_OK`, **se permite marchar**) y al pulsarlo cae a 0.

El ESP32 trae resistencias internas (`INPUT_PULLUP`), pero aquí **todas son externas** y el programa usa `INPUT`. Razón: la línea de PARO también irá a la compuerta 74HC08, que no tiene resistencias internas. Corriente con el contacto cerrado: $3{,}3\,\mathrm{V}/10\,\mathrm{k} = 0{,}33\,\mathrm{mA}$, despreciable.



(a) MARCHA
pull-down - pulsado = 1

(b) PARO
pull-up - reposo = 1 = `PARO_OK`

(c) DIP, se repite 3 veces
pull-down - ON = 1

Figura 4: Las cinco entradas. Cada una tiene su propia resistencia de $10\,\mathrm{k}$.

1. Con el USB desconectado, arme la Figura 4. En los pulsadores de cuatro patas, **los dos pines de cada costado pueden estar unidos internamente**; compruebe la orientación con el multímetro y sitúelos cruzando el canal central. Para cada DIP, confirme cuáles terminales se conectan al pasar a ON.
2. Conecte el USB y escriba 2. El monitor imprime cada cambio y el LED rojo repite el pulsador MARCHA.
3. Llene la tabla accionando cada elemento y **soltándolo**. Compruebe también B y FALLA en el Monitor serie; sus valores esperados son los mismos que para A.

Entrada	Valor en reposo	Valor accionado	Al soltar, ¿vuelve sola?
MARCHA (pulsador)	_____	_____	_____
PARO (pulsador)	_____	_____	_____
A (DIP)	_____	_____	_____

Prueba una falla

Desconecte el USB y **retire solamente la resistencia de $10\,\mathrm{k}$ de A**. Conecte el USB, deje A en OFF y acerque el dedo al cable sin tocar los contactos. Observe si hay lecturas inestables (puede que no ocurran en su montaje). **Desconecte de nuevo el USB y reponga la resistencia antes de continuar.**

Preguntas. (1) ¿Cuál de las dos entradas **recuerda** la posición: el pulsador o el interruptor? (2) ¿Por qué PARO vale 1 cuando nadie lo toca?

4 Marcha y paro con retención

Objetivo

Hacer que un pulsador de MARCHA deje el equipo encendido (retención) y que PARO lo apague **aunque la orden del programa permanezca activa**.

La **retención** significa que una pulsación breve de MARCHA mantiene una orden activa incluso después de soltar el botón. En un circuito clásico esto se hace con un contacto auxiliar; aquí el programa del ESP32 **recuerda** el estado mediante una variable. La expresión siguiente representa la condición de marcha: el signo $+$ significa

O, el punto significa *Y* y la barra superior significa *NO*.

$$\text{KM1} = (\text{MARCHA} + \text{KM1}) \cdot \text{PARO_OK} \cdot \overline{\text{FALLA}}$$

Dos barreras con funciones diferentes: el programa decide si debe activar KM1, pero el cableado impide que llegue esa orden cuando se pulsa PARO. La orden del ESP32 (`CMD1`) no enciende el LED directamente: pasa por una **compuerta Y** del 74HC08 cuya otra entrada es PARO_OK. Este circuito solo realiza inhibición didáctica de órdenes: no es un paro de emergencia de seguridad certificada. La compuerta solo da 1 si las dos entradas son 1. Si se pulsa PARO, la salida cae a 0 por cable aun con CMD1 alto, siempre que el 74HC08 y el cableado funcionen correctamente.

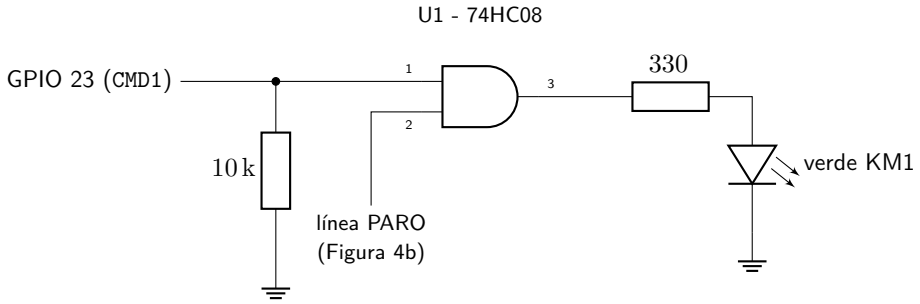


Figura 5: La compuerta Y deja pasar la orden del ESP32 solo si PARO está en reposo. La resistencia de $10\,\text{k}$ mantiene `CMD1` en 0 mientras el ESP32 arranca.

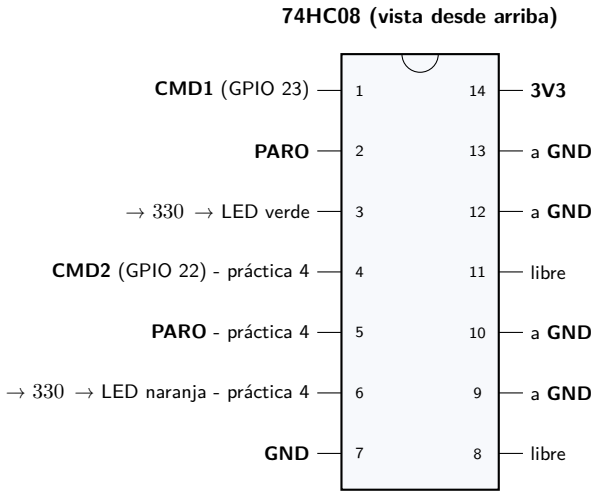


Figura 6: Conexiones del integrado. Ponga el condensador de $100\,\text{nF}$ entre los pines 14 y 7. Las entradas que no se usan (9, 10, 12, 13) van a `GND`: tampoco pueden quedar al aire. Hasta la práctica 4, conecte también los pines 4 y 5 a `GND`.

1. Con el USB desconectado, coloque el 74HC08 atravesando el canal central, con la muesca hacia arriba. Arme la Figura 6 y luego la Figura 5.
2. Pida al instructor que revise: pin 14 a 3V3, pin 7 a `GND`, condensador cerámico de $100\,\text{nF}$ lo más cerca posible del integrado (no tiene polaridad) y **ausencia de cortocircuitos**. Use un **74HC08** compatible con 3,3 V; no lo sustituya automáticamente por un 74LS08.
3. Escriba 3 y realice los eventos de la tabla, en orden. El LED rojo indica FALLA.

Evento	LED verde: predigo	LED verde: observo
Pulsar y soltar MARCHA	_____	_____
Pulsar y soltar PARO	_____	_____
MARCHA, luego FALLA en ON y otra vez en OFF	_____	_____
Mantener PARO pulsado y pulsar MARCHA	_____	_____
Escribir `b` (programa **trabado**, `CMD1` $= 1$ siempre) y pulsar PARO	_____	_____

Preguntas. (1) ¿Qué hace que el LED siga encendido después de soltar MARCHA? (2) Después de una FALLA, ¿por qué hay que pulsar MARCHA otra vez? (3) Con el programa trabado, ¿quién apagó el LED: el ESP32 o la compuerta? ¿Por qué eso importa en un tablero real?

5 Transferencia red–planta

Objetivo

Usar los mismos elementos para pasar la carga de la red a la planta: confirmar la falla, esperar, transferir y volver.

Ahora el DIP representa las fuentes: **A = red apta**, **B = planta disponible**. El LED verde es KM1 (carga en red), el naranja es KM2 (carga en planta) y el rojo es la alarma. El ESP32 aplica los tiempos del caso, reducidos para el aula:

La red debe faltar $3\,\mathrm{s}$ seguidos	para confirmar la falla (un microcorte no transfiere)
$2\,\mathrm{s}$ sin ninguna orden KM1/KM2	tiempo muerto antes de cerrar la otra salida
La red debe volver $5\,\mathrm{s}$ seguidos	para iniciar el retorno a KM1 (un retorno inestable no cuenta)

Qué debe observar con el programa de referencia: si A pasa a OFF y B sigue en ON, KM1 se desconecta tras $3\,\mathrm{s}$ y KM2 se conecta $2\,\mathrm{s}$ después (**aproximadamente 5 s desde la caída**). Si A vuelve a ON mientras se alimenta desde B, se esperan $5\,\mathrm{s}$ de red estable, se desconecta KM2 y pasan $2\,\mathrm{s}$ antes de conectar KM1 (**aproximadamente 7 s**). Los tiempos pueden variar ligeramente por la lectura periódica del controlador. El banco **no mide la tensión** de ninguna fuente: los DIP representan estados simulados.

PARO en la práctica 4: la compuerta AND apaga físicamente ambos indicadores mientras se mantiene pulsado. El programa también pasa a un estado detenido; para reiniciar, libere PARO y escriba `4` de nuevo. El fallo forzado con `k` demuestra que las dos compuertas AND **no** impiden órdenes simultáneas entre KM1 y KM2.

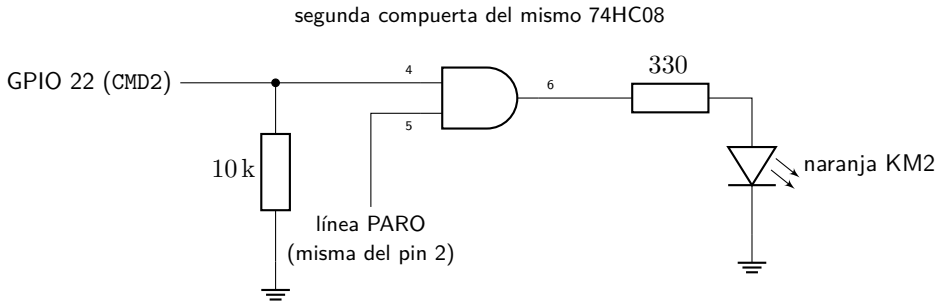


Figura 7: Se agrega la segunda compuerta: igual a la Figura 5, con pines 4, 5 y 6. Retire antes los jumpers que llevaban los pines 4 y 5 a `GND`.

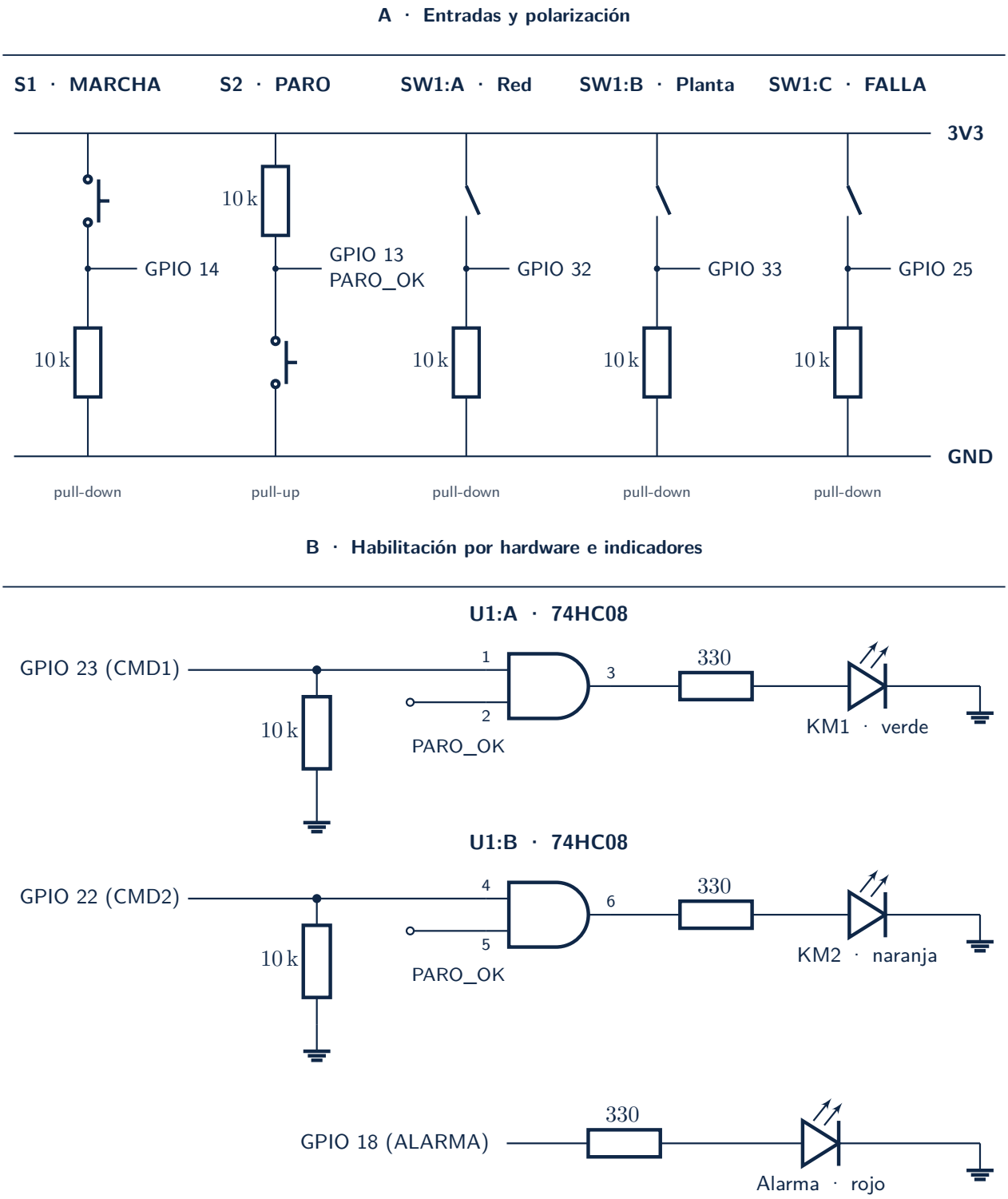
1. Con el USB desconectado, arme la Figura 7.

2. Conecte el USB, ponga A = ON, B = ON y PARO liberado, y escriba `4`. Debe encender el verde (KM1).
3. Realice los eventos de la tabla. Use el cronómetro del celular y lea los mensajes del monitor.

Evento	**Predigo**	**Observo**
Falla de red: A en OFF. ¿Cuánto tarda en encender el naranja?	_____	_____
Retorno: A en ON. ¿Cuánto tarda en volver el verde?	_____	_____
Microcorte: A en OFF y en ON antes de $3\,\mathrm{s}$. ¿Transfiere?	_____	_____
Sin planta: B en OFF y luego A en OFF. ¿Qué LED enciende?	_____	_____
PARO pulsado en cualquier momento. ¿Qué pasa con los LEDs?	_____	_____

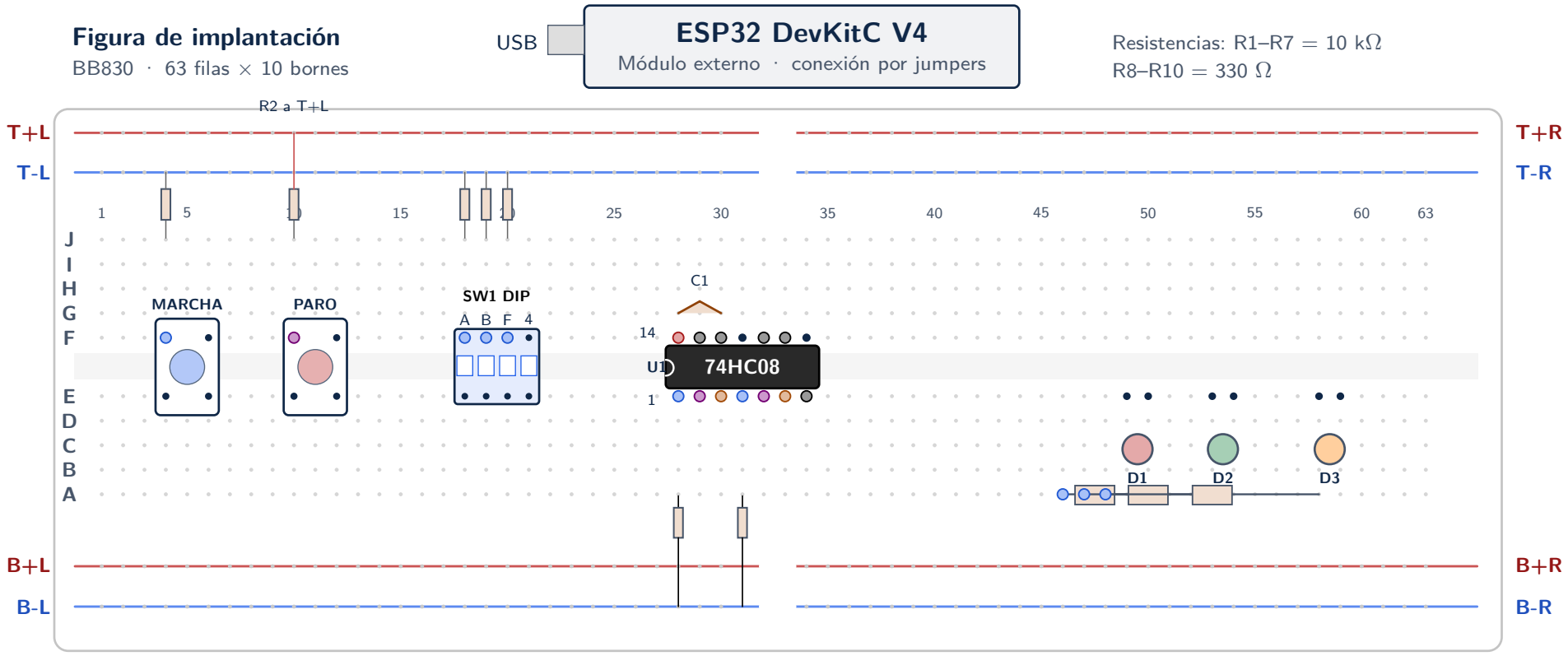
Prueba una falla

Con A = ON, B = ON y PARO liberado, escriba `k`: el programa de referencia simula un error y ordena KM1 y KM2 a la vez. ¿Qué ve? En un tablero real, esa condición sería inadmisible y requeriría enclavamientos independientes. Aquí solo se encienden LED indicadores. El control normal evita solapes, pero esta prueba fuerza dos órdenes simultáneas y evidencia que el PARO común no constituye un enclavamiento cruzado. **Discuta en pareja:** ¿qué tendría que agregarse *por cable* para que sea imposible? (Pista: el módulo *Enclavamientos* del laboratorio interactivo.) Escriba `k` otra vez: debe desaparecer la simultaneidad y la lógica debe volver al estado normal (con A = ON, solo verde).



U1: pin 14 a 3V3, pin 7 a GND y C1 = 100 nF entre ambos, junto al integrado. Entradas de compuertas no usadas (9, 10, 12 y 13) a GND; salidas 8 y 11 sin conexión.

Figura 8: Integración eléctrica del banco didáctico. Los nombres de red identifican las conexiones al ESP32 y al 74HC08; cada conductor se interpreta según los nodos dibujados. Las salidas son indicadores LED, no contactos de potencia.



01 · Entradas

S1 MARCHA, S2 PARO y SW1 DIP, en el extremo izquierdo. R2 de PARO se conecta a 3V3; las demás entradas, a GND.

02 · Compuertas

U1 74HC08 cruza el canal central, filas 28–34. C1 de $100\,\mathrm{nF}$ desacopla los pines 14 y 7.

03 · Indicadores

D1 alarma, D2 KM1 y D3 KM2, a la derecha. Cada LED incorpora una resistencia serie de $330\,$.

Figura 9: Plano de implantación BB830 con referencias de componentes y orificios. S1: MARCHA; S2: PARO; SW1: DIP; U1: 74HC08; D1–D3: indicadores. Este dibujo es una guía de ubicación, no muestra todos los jumpers. Cablee según el esquema y la tabla de conexiones del Anexo A; verifique los rieles interrumpidos.

Cierre

Evidencia que debe entregar

1. Una foto del montaje final con el LED verde encendido.
2. Las cuatro tablas de esta guía llenas.
3. Respuestas breves (2 o 3 líneas cada una):
   - ¿Por qué el LED lleva 330 y no otro valor?
   - ¿Qué es una entrada flotante y cómo la evitó en este banco?
   - ¿Por qué el PARO pasa por la compuerta y no solo por el programa?

A Tabla completa de conexiones

Esta tabla complementa los esquemas y evita confundir el *plano de ubicación* de la protoboard con un plano de cableado. Cada conexión se realiza con **USB desconectado**. Los rieles positivo y negativo son comunes a todo el montaje.

Elemento/señal	Unir estos terminales	Comprobación / función
Alimentación	ESP32 `3V3` a riel positivo; ESP32 `GND` a riel negativo	Solo 3,3 V; verificar continuidad de rieles
S1 MARCHA	un lado a 3V3; el otro a GPIO 14 y resistencia R1 de 10 kΩ a GND	Reposo: 0; pulsado: 1
S2 PARO	un lado a GND; el otro a GPIO 13, 74HC08 pines 2 y 5 y resistencia R2 de 10 kΩ a 3V3	Reposo: 1; pulsado: 0 (PARO_OK)
SW1:A (red apta)	un lado a 3V3; el otro a GPIO 32 y R3 de 10 kΩ a GND	ON: 1; OFF: 0
SW1:B (planta disponible)	un lado a 3V3; el otro a GPIO 33 y R4 de 10 kΩ a GND	ON: 1; OFF: 0
SW1:C (FALLA)	un lado a 3V3; el otro a GPIO 25 y R5 de 10 kΩ a GND	ON: 1; OFF: 0
74HC08 alimentación	pin 14 a 3V3; pin 7 a GND; C1 = 100 nF entre pines 14 y 7	C1 junto a U1, no polarizado
CMD1	ESP32 GPIO 23 a U1 pin 1 y R6 de 10 kΩ a GND	Entrada primera AND: evita orden indefinida al arranque
CMD2	ESP32 GPIO 22 a U1 pin 4 y R7 de 10 kΩ a GND	Entrada segunda AND
KM1 / D2 verde	U1 pin 3 → resistencia 330 Ω → ánodo LED verde; cátodo a GND	Salida primera AND
KM2 / D3 naranja	U1 pin 6 → resistencia 330 Ω → ánodo LED naranja; cátodo a GND	Salida segunda AND
Alarma / D1 rojo	ESP32 GPIO 18 → resistencia 330 Ω → ánodo LED rojo; cátodo a GND	Salida directa del ESP32
Puertas no usadas	U1 pines 9, 10, 12 y 13 a GND; pines 8 y 11 libres	No dejar entradas del chip flotantes

Seguridad y cuidado del equipo

Antes de conectar el USB: revise orientación de U1 (muesca y pin 1), polaridad de los tres LEDs, continuidad de GND, valores de resistencias y ausencia de cortos entre los rieles. La ubicación física de resistencias/LEDs depende de la protoboard: respete las **redes eléctricas** de la tabla, no solamente las posiciones de la lámina.

B Programa de referencia del ESP32

El siguiente programa es una **implementación didáctica de referencia** desarrollada para que esta guía sea autocontenida. Si el instructor dispone de una versión oficial del curso, debe comprobar que sus mensajes y temporizaciones coincidan antes de utilizarla. Copie el contenido a un archivo llamado `banco_simple_esp32.ino`, cárguelo al ESP32 y configure el Monitor serie a 115200 baudios.

Listing 1: Programa autónomo de referencia para las cuatro prácticas

```
/*
  Banco basico ATSEP: ESP32 + 74HC08 (referencia didactica).
  Solo LEDs a 3,3 V. Nunca conectar cargas o tensiones de red.
  Monitor serie: 115200 baudios; comandos 1, 2, 3, 4, b, k.
*/
const uint8_t LED_ALARMA = 18;
const uint8_t PIN_MARCHA = 14;
const uint8_t PIN_PARO = 13; // 1 = PARO_OK, 0 = pulsado
const uint8_t PIN_A = 32; // red apta
const uint8_t PIN_B = 33; // planta disponible
const uint8_t PIN_FALLA = 25;
const uint8_t CMD1 = 23; // a 74HC08 pin 1
const uint8_t CMD2 = 22; // a 74HC08 pin 4

const unsigned long CONF_FALLA = 3000UL;
const unsigned long TIEMPO_MUERTO = 2000UL;
const unsigned long CONF_RETORNO = 5000UL;

enum Etapa { RED, ESPERA_FALLA, MUERTO_PLANTA,
           PLANTA, ESPERA_RETORNO, MUERTO_RED,
           SIN_FUENTE, DETENIDO };
Etapa etapa = DETENIDO;
unsigned long inicioEtapa = 0;
int practica = 0;
bool retenido = false;
bool anteriorMarcha = false;
bool trabado = false; // prueba b: CMD1 forzado alto
bool fallaDoble = false; // prueba k: CMD1 y CMD2 altos
int anteriorEntradas = -1;

bool paroOK() { return digitalRead(PIN_PARO) == HIGH; }
bool marcha() { return digitalRead(PIN_MARCHA) == HIGH; }
bool redApta() { return digitalRead(PIN_A) == HIGH; }
bool plantaLista() { return digitalRead(PIN_B) == HIGH; }
bool hayFalla() { return digitalRead(PIN_FALLA) == HIGH; }

void salidas(bool km1, bool km2, bool alarma) {
  // La compuerta 74HC08 inhibe los LEDs KM1/KM2 al pulsar PARO.
  digitalWrite(CMD1, km1 ? HIGH : LOW);
  digitalWrite(CMD2, km2 ? HIGH : LOW);
  digitalWrite(LED_ALARMA, alarma ? HIGH : LOW);
}

void cambiar(Etapa nueva, unsigned long ahora, const char* mensaje) {
  etapa = nueva;
  inicioEtapa = ahora;
  Serial.println(mensaje);
}

void seleccionar(int p, unsigned long ahora) {
  practica = p;
  retenido = false;
  trabado = false;
```

```
  fallaDoble = false;
  anteriorMarcha = marcha();
  anteriorEntradas = -1;
  salidas(false, false, false);
  Serial.printf("Practica %d seleccionada.\n", p);
  if (p == 4) {
    if (!paroOK()) cambiar(DETENIDO, ahora, "PARO activo: libere y escriba 4.");
    else if (redApta()) cambiar(RED, ahora, "RED: KM1 habilitado.");
    else cambiar(SIN_FUENTE, ahora, "Sin red: esperando fuente.");
  }
}

void procesarComandos(unsigned long ahora) {
  while (Serial.available() > 0) {
    char c = (char)Serial.read();
    if (c >= '1' && c <= '4') seleccionar(c - '0', ahora);
    else if (c == 'b' && practica == 3) {
      trabado = !trabado;
      Serial.println(trabado ? "Prueba b: CMD1 forzado a 1."
                             : "Prueba b finalizada.");
    } else if (c == 'k' && practica == 4) {
      fallaDoble = !fallaDoble;
      Serial.println(fallaDoble ? "Prueba k: ambas ordenes a 1."
                                : "Prueba k terminada: control normal.");
    }
  }
}

void reportarEntradas() {
  int bits = (marcha() ? 1 : 0) | (paroOK() ? 2 : 0)
           | (redApta() ? 4 : 0) | (plantaLista() ? 8 : 0)
           | (hayFalla() ? 16 : 0);
  if (bits != anteriorEntradas) {
    anteriorEntradas = bits;
    Serial.printf("MARCHA=%d PARO_OK=%d A=%d B=%d FALLA=%d\n",
      marcha(), paroOK(), redApta(), plantaLista(), hayFalla());
  }
}

void controlarTransferencia(unsigned long ahora) {
  if (!paroOK()) {
    if (etapa != DETENIDO)
      cambiar(DETENIDO, ahora, "PARO: control detenido; reinicie con 4.");
    return;
  }
  switch (etapa) {
    case RED:
      if (!redApta()) cambiar(ESPERA_FALLA, ahora, "Fallo de red: contando 3 s.");
      break;
    case ESPERA_FALLA:
      if (redApta()) cambiar(RED, ahora, "Microcorte: sigue KM1.");
      else if (ahora - inicioEtapa >= CONF_FALLA)
        cambiar(MUERTO_PLANTA, ahora, "Red confirmada perdida: KM1 abierto.");
      break;
    case MUERTO_PLANTA:
      if (ahora - inicioEtapa >= TIEMPO_MUERTO) {
        if (redApta()) cambiar(RED, ahora, "Red disponible: KM1.");
        else if (plantaLista()) cambiar(PLANTA, ahora, "Planta: KM2 habilitado.");
        else cambiar(SIN_FUENTE, ahora, "Ninguna fuente disponible.");
      }
      break;
    case PLANTA:
      if (!plantaLista()) cambiar(SIN_FUENTE, ahora, "Planta no disponible: KM2 abierto.");
      else if (redApta()) cambiar(ESPERA_RETORNO, ahora, "Red retorno: contando 5 s.");
      break;
    case ESPERA_RETORNO:
      if (!redApta()) cambiar(PLANTA, ahora, "Retorno inestable: sigue KM2.");
      else if (!plantaLista()) cambiar(SIN_FUENTE, ahora, "Planta perdida: KM2 abierto.");
      else if (ahora - inicioEtapa >= CONF_RETORNO)
        cambiar(MUERTO_RED, ahora, "Retorno confirmado: KM2 abierto.");
      break;
    case MUERTO_RED:
      if (ahora - inicioEtapa >= TIEMPO_MUERTO) {
        if (redApta()) cambiar(RED, ahora, "Red: KM1 habilitado.");
        else if (plantaLista()) cambiar(PLANTA, ahora, "Sin red: vuelve KM2.");
```

```
      else cambiar(SIN_FUENTE, ahora, "Sin fuentes.");
    }
    break;
  case SIN_FUENTE:
    if (redApta()) cambiar(RED, ahora, "Red disponible: KM1.");
    else if (plantaLista()) cambiar(MUERTO_PLANTA, ahora, "Planta lista: espera 2 s.");
    break;
  case DETENIDO:
    break; // solo se rearma enviando 4
 }
}

void setup() {
 Serial.begin(115200);
 pinMode(PIN_MARCHA, INPUT); // resistencias externas de 10 kOhm
 pinMode(PIN_PARO, INPUT);
 pinMode(PIN_A, INPUT);
 pinMode(PIN_B, INPUT);
 pinMode(PIN_FALLA, INPUT);
 pinMode(LED_ALARMA, OUTPUT);
 pinMode(CMD1, OUTPUT);
 pinMode(CMD2, OUTPUT);
 salidas(false, false, false);
 Serial.println("Banco básico ATSEP. Escriba 1, 2, 3 o 4.");
}

void loop() {
 unsigned long ahora = millis();
 procesarComandos(ahora);
 bool m = marcha();
 bool ok = paroOK();
 if (practica == 1) salidas(false, false, true);
 else if (practica == 2) {
   reportarEntradas();
   salidas(false, false, m);
 } else if (practica == 3) {
   if (!ok || hayFalla()) retenido = false;
   else if (m && !anteriorMarcha) retenido = true;
   salidas(trabado || retenido, false, hayFalla());
 } else if (practica == 4) {
   controlarTransferencia(ahora);
   bool km1 = etapa == RED || etapa == ESPERA_FALLA;
   bool km2 = etapa == PLANTA || etapa == ESPERA_RETORNO;
   if (fallaDoble) { km1 = true; km2 = true; }
   salidas(km1, km2, !redApta() || hayFalla());
 } else salidas(false, false, false);
 anteriorMarcha = m;
 delay(10);
}
```